

6 01/09/2021

6.1 Traccia da 12 punti

6.1.1 2pt

Siano dati due vettori di interi positivi $v1$ e $v2$ di dimensioni $d1$ e $d2$. Si scriva una funzione in grado di generare una matrice di dimensioni $d1 \times d2$ in cui il contenuto della generica cella $[i, j]$ sia il prodotto dell'elemento di posizione i di $v1$ e dell'elemento di posizione j di $v2$. Il prototipo della funzione sia quello presentato a seguire. L'allocazione della matrice deve avvenire dentro alla funzione. La matrice creata e i suoi contenuti devono essere disponibili per chi invoca la funzione. Il prototipo va completato opportunamente in modo da poter rispondere alle specifiche della richiesta.

```
void f(int *v1, int *v2, int d1, int d2, ...);
```

Esempio

$$v1 = \{2, 4, 6\}$$
$$v2 = \{1, 3, 5, 7\}$$
$$M = \begin{pmatrix} 2 & 6 & 10 & 14 \\ 4 & 12 & 20 & 28 \\ 6 & 18 & 30 & 42 \end{pmatrix}$$

6.1.2 4pt

Definire una struttura dati adeguata a rappresentare una lista doppio linkata di interi come ADT I classe, dove il tipo lista si chiami **LIST**. **La lista definita non deve fare uso di sentinelle.**

Indicare esplicitamente in quale modulo/file appare la definizione dei tipi proposti. La struttura così definita è quella da usare per completare la seconda domanda di questo esercizio. **Non è ammesso l'uso di funzioni di libreria.**

Implementare una funzione caratterizzata dal seguente prototipo:

```
void purge(LIST l, int div)
```

la funzione **purge** cancella dalla lista tutti i nodi che contengono un numero non divisibile per **div**.

Non è ammesso l'uso di funzioni di libreria. Attenersi al prototipo dato.

Esempio

$\text{div} = 3$

$L = 9 \leftrightarrow 8 \leftrightarrow 7 \leftrightarrow 6 \leftrightarrow 5 \leftrightarrow 4 \leftrightarrow 3 \leftrightarrow 2 \leftrightarrow 1 \leftrightarrow 0$

$L' = 9 \leftrightarrow 6 \leftrightarrow 3 \leftrightarrow 0$

6.1.3 6pt

Data una funzione wrapper con il seguente prototipo

```
void solve(char *target, part *P, int nParts);
```

Dove **target** è una stringa obiettivo da costruire sfruttando le stringhe disponibili nel vettore di strutture **P**, di tipo **part**, lungo **nParts**. **part** è una struttura definita come segue:

```
typedef struct {  
    char *s;  
    int pos;  
    int costo;  
} part;
```

usata per rappresentare una possibile *parte* della stringa obiettivo. Una parte è caratterizzata da una sottostringa **s**, che può essere posizionata solo a partire dal carattere di posizione **pos**. Ogni parte ha un certo costo associato. Scrivere una funzione ricorsiva che determini se è possibile generare la stringa **target** sfruttando le stringhe disponibili in **P** minimizzando il costo totale di costruzione. A parità di costo prediligere la soluzione che fa uso di più parti. Giustificare la scelta del modello combinatorio adottato. Giustificare la scelta del/dei criteri di pruning adottati, o il motivo della loro assenza.

Esempio

target = "persona"

P = {
 {"p", 0, 1}, // p0
 {"pers", 0, 5}, // p1
 {"er", 1, 4}, // p2
 {"ers", 1, 4}, // p3
 {"sa", 3, 1}, // p4

```
{"so", 3, 2}, // p5  
{"ato", 0, 1}, // p6  
{"on", 4, 2}, // p7  
{"ona", 4, 3}, // p8  
{"a", 6, 1} // p9  
}
```

Usando p0, p3, p7, p9: costo 8, cardinalità soluzione 4 (soluzione ottima).

Usando p0, p3, p8: costo 8, cardinalità soluzione 3.

Usando p1, p8: costo 8, cardinalità soluzione 2.

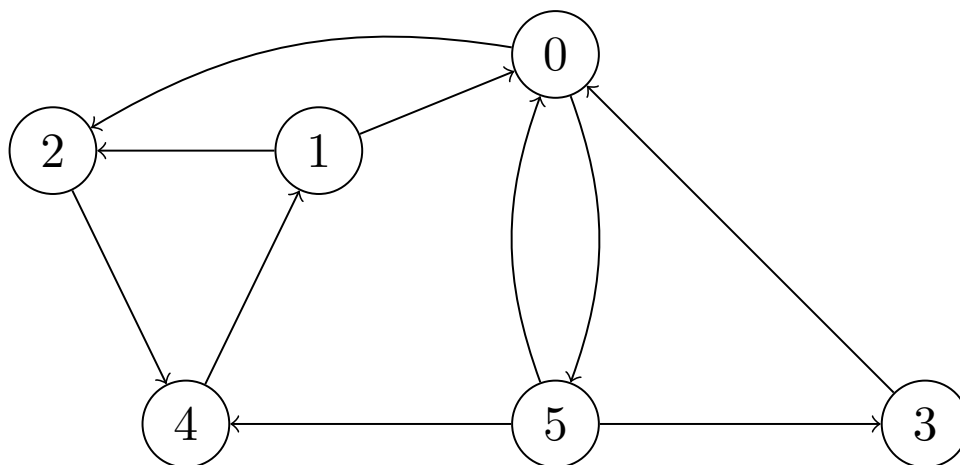
6.2 Traccia da 18 punti

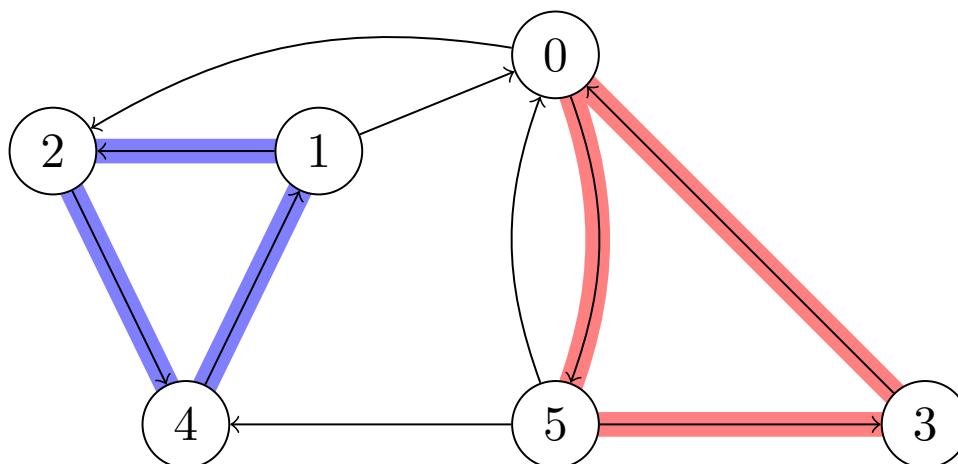
6.2.1 Breve introduzione e definizioni

Dato un grafo $G = (V, E)$ orientato e connesso, si definisce *copertura ciclica* un insieme di cicli la cui unione copre tutti i vertici del grafo. Perché un vertice sia ritenuto coperto, deve quindi far parte di almeno uno dei cicli inclusi nell'insieme preso in considerazione. Nel contesto dell'appello corrente, quando si parla di cicli, si intende sempre e solo **cicli semplici**. Una copertura può essere *sovrapposta*, se lo stesso vertice appare in più cicli, o *disgiunta*, nel caso contrario.

Esempio

A seguire è riportato un grafo di esempio e una sua copertura disgiunta che fa uso di due cicli.





Il grafo è memorizzato in un file testuale `grafo.txt` organizzato come segue:

- Sulla prima riga appare una coppia di interi V e E a rappresentare il numero di vertici e archi del grafo;
- Seguono E righe, ognuna della quale riporta una coppia $v\ w$ a rappresentare i singoli archi del grafo;
- Si assuma che i vertici siano identificati univocamente con un valore intero nell'intervallo $0 \dots V-1$.

L'insieme dei cicli è anch'esso fornito, ed è memorizzato in un file testuale `cicli.txt` organizzato come segue:

- I cicli sono riportati in ragione di uno per riga. Il numero di cicli non è noto a priori;
- Ogni i -esima riga presenta un primo valore intero `len_i`, rappresentante la lunghezza del ciclo, seguito da `len_i` valori interi a rappresentare i vertici che compongono il ciclo stesso.

A seguire sono riportati due file completi che riprendono i contenuti dell'esempio presentato in precedenza.

`grafo.txt`

```
6 10
0 2
0 5
1 0
1 2
2 4
```

```
3 0
4 1
5 0
5 3
5 4
```

cicli.txt

```
4 0 5 4 1
3 0 5 3
2 0 5
4 0 2 4 1
3 1 2 4
```

6.2.2 Strutture dati

Acquisire in una opportuna struttura dati idonea i contenuti del file `grafo.txt`. Se si fa uso della struttura grafo ADT vista a lezione, riportare le eventuali modifiche apportate alla sua definizione. Definire una struttura dati idonea a memorizzare un numero arbitrario di cicli e implementare le funzioni opportune per acquisire i contenuti del file `cicli.txt`.

6.2.3 Problema di verifica

Dato un elenco ordinato di vertici, in un formato a discrezione del candidato, verificare se questo rappresenta un ciclo.

6.2.4 Problema di ricerca e ottimizzazione

Individuare, se possibile, un insieme a cardinalità minima di cicli, scelti dall'elenco letto in precedenza, la cui unione copre tutti i vertici del grafo. A parità di cardinalità, prediligere una copertura disgiunta.